

Trader Edition

Install and maintain the dashboard source with all six strategy frameworks.

Release 2.1.0-source.20260928 | Updated October 3, 2026

Scope and limitations

This is downloadable source software. Hosting, trading funds and external services are separate. Build and signal checks are not certification for unattended real-money execution. Complete venue and recovery validation for your installation.

Core includes Darvas. Trader includes Core and all six strategies. Add-ons need the 2.1 plugin loader and are not compatible with the browser-only 2.0 template.

Requirements

Node 22 LTS, version 22.3 or newer, as used by this release; npm and the shipped lockfile.

A dedicated Supabase project with Auth, REST and PostgreSQL; plain PostgreSQL alone is insufficient.

PostgreSQL client tools with psql on PATH, a verified owner account and private environment configuration.

For remote use: an authenticated HTTPS proxy, firewall, monitoring and tested backups.

Purchase and download

1. Sign in to the account used at checkout. Open Account and locate the confirmed order. If payment is processing, wait for fulfillment rather than buying again.
2. Generate a fresh product Download link. Save the ZIP, receipt, release version and checksum. Expired links can be regenerated from the account.
3. Compare the archive SHA-256 with the release recorded below. Extract into a new directory; retain the original archive for rollback.
4. Open README.md, RUNBOOK.md or INSTALL.md, manifest.json where included, and LICENSE.md before changing an installation.

Archive SHA-256: bd2ca79878d18b509732a8e584d9107a2ce0d952a8becd0a76a893c1dfdc8798

Installation

1. Extract the edition into a new directory and copy .env.example to .env.local. On Windows use Copy-Item; on Linux/macOS use cp.
2. Create and verify the owner in Supabase Auth. Configure ACCOUNT_OWNER_EMAIL, REQUIRE_AUTH=1, ADMIN_TOKEN, DATABASE_URL and the exact Supabase settings in the environment example. Keep

server-role credentials server-only.

3. Keep `TRADING_MODE=paper` and signing keys blank during installation. Review the schema and migrations, then run the commands below. `db:init` installs atomically, checks migration integrity and refuses automatic owner transfer.

4. Start the app and open `http://localhost:9005`. Sign in as the owner. Verify anonymous and wrong-owner access is refused, and confirm settings persist after a restart.

5. Create a paused, unfunded agent from the installed strategies. Choose a market and interval, then configure position limits, allocation and drawdown controls before testing.

Run from the dashboard root

```
npm ci
```

```
npm run db:init
```

```
npm run typecheck
```

```
npm test
```

```
npm run build
```

```
npm start
```

Step-by-step practice tutorial

1. Install and verify access

Finish the commands above. Sign in as the configured owner, then test a separate signed-out session. Expected result: your workspace loads, but anonymous access is rejected.

2. Create the first paused agent

Open Agents and choose an installed strategy. Name it clearly, select one market/interval and leave it paused with no funds allocated. Expected result: the saved strategy and status agree after reload.

3. Read the first decision

Use a verified paper/research path with completed candles. Record the timestamp, direction, confidence and reasoning. Expected result: a traceable signal or a valid neutral decision; no live order during this exercise.

4. Check all six installed modules

Verify darvas-box, elliot-wave, vwap-breakout, heikin-ashi, mean-reversion and macro-sentiment in the loader. Test each separately before combining budgets. Trader supplies six frameworks; it does not imply six independently safe simultaneous venue positions.

5. Configure risk and ownership

Review agent and farm budgets together, maximum exposure and drawdown limits. Identify which runtime owns each position. Expected result: allocations are not counted as new deposits and duplicate strategy ownership is avoided.

6. Prove persistence and recovery

Pause, save settings, restart and compare the same agent and configuration. Restore a backup only in isolation. Expected result: ownership and settings survive; restoration does not create duplicate live engines.

Acceptance checklist

- [] Archive checksum and version match the guide.
- [] Install completes without suppressing errors; the module loads from its declared path.
- [] Correct owner, market, network and candle interval are recorded.
- [] Decision timestamp, direction, reasoning and parameters are captured; neutral is accepted.
- [] Settings and ownership persist after restart; backup restore is tested in isolation.
- [] Before live execution: entry, fill, protection, modification, exit, fees and P&L reconcile; partial fills, cancellation failure and lost responses are tested.

Read RELEASE-NOTES.md for historical acceptance evidence. The shipped test:isolation and test:runtime commands are developer fixtures restricted to a disposable local Supabase URL (127.0.0.1:55521); they are not general customer production checks.

Maintain and customize

Keep a copy of the immutable ZIP, lockfile, private configuration and a database backup before an update. Store encrypted backups separately and prove a restore in an isolated compatible instance.

Pause new entries and reconcile existing venue orders and positions before maintenance. Pausing entries is not the same as closing positions. Removing a plugin does not close exchange positions.

Monitor the last successful cycle, data age, rejected orders, protection status, reconciliation gaps, database backups and disk space. Missing telemetry is an unknown state.

For rollback, stop the new runtime, reconcile the venue, restore a compatible application/schema pair in isolation and verify ownership and protection before resuming. A database restore cannot reverse a trade or transfer.

Customize readable source in a separate branch. Preserve the original defaults, rebuild any changed compiled module, and test the same inputs before and after. Follow LICENSE.md; source ownership does not imply unrestricted redistribution.

Troubleshooting

Payment confirmed but download missing

Check order status in Account and contact support with the order reference if fulfillment is delayed. Do not make another purchase to repair access.

Download link expired

Generate a fresh link from the purchasing account. The documentation downloads are public; the paid software archive remains entitlement-protected.

Strategy absent from the selector

Confirm manifest.json is directly inside the manifest installPath, the entryPoint exists, compatibility matches and the loader has no errors. Restart after installation; do not add another nesting level.

No trade despite available capital

Inspect the latest decision, candle history and freshness, symbol, interval, network, existing positions and risk rejection. Neutral is a valid result.

Build or database initialization fails

Keep the exact failed command and redacted diagnostic. Check Node, psql, Supabase settings and owner verification. Do not bypass types, authentication or schema integrity checks.

Balances disagree or an order outcome is unknown

Compare the same account and network at the venue, including fills, fees, margin and open orders. Reconcile before retrying; a lost response does not mean the order failed.

Help and supporting video lessons

<https://www.civalsystems.com/docs/products/multi-strat-bundle>

Supporting videos show the broader demo interface. The product-specific written tutorial matches this archive.

Contact <https://www.civalsystems.com/contact> with your order reference, product/version, OS, Node version and redacted error. Never send secrets. Refund requests: <https://www.civalsystems.com/refund-request>.

Source references

.env.example

LICENSE.md

README.md

RELEASE-NOTES.md

RUNBOOK.md

package.json